Kubernetes, which is another favourite choice.

For more detailed information about Docker Swarm and its various commands, do refer to the links given below:

- *https://docs.docker.com/engine/swarm/*
- *https://docs.docker.com/engine/reference/commandline/swarm/*

Running applications in the Docker Swarm environment

Before we explore how to run applications inside the Docker Swarm environment, we need to build the images that are stored in Docker Hub or those present in the local registry. Once we have the images identified and ready, Docker Compose needs to be built with various components.

Here we have taken the Jmeter application as an example. The images have been built using Jmeter and its required resources, which are stored with the name 'ubuntujmeter' in the host machine.

Jmeter is a popular open source load testing tool. Each Jmeter instance generates a certain load on to the system under test. By making it replicable, Docker enables sharing the JMeter tests between users and replicating the test environment.

The following is the Docker Compose file that we have created for this scenario:

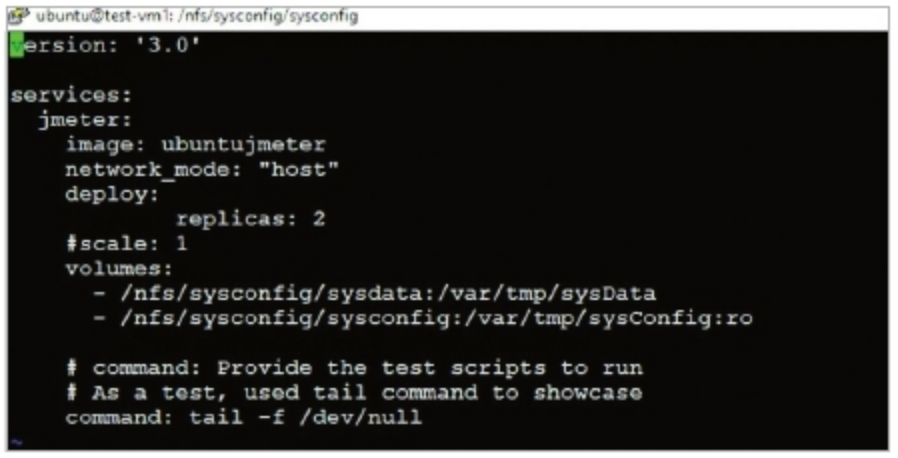

Figure 1: Docker Compose file

```
version: '3.0'

services:
  jmeter:
    image: ubuntujmeter
    network_mode: "host"
    deploy:
            replicas: 2
    #scale: 1
    volumes:
      - /nfs/sysconfig/sysdata:/var/tmp/sysData
      - /nfs/sysconfig/sysconfig:/var/tmp/sysConfig:ro

    # command: Provide the test scripts to run
    # As a test, used tail command to showcase
    command: tail -f /dev/null
```

In the above file, the *deploy* command has the parameter *replicas*, which defines the default value of the number of instances needed to be run while starting the services. Here we have specified two instances to be started in two different nodes. Do check out the *Docker-compose* file command reference for the rest of the content in the file at *https://docs.docker.com/compose/overview/*.

Now, execute the following command to start the services:

```
# sudo docker stack deploy --compose-file docker-compose.yml
<servicename>
```

Here, the service name is *stackjmeter.*

Figure 2: The Docker stack deploy command

In the above configuration, we can easily verify that two instances have been started. Use the following command to verify this:

```
# sudo docker service ls
```

Now that the instances have been started in both the nodes and they are ready, log in to both the nodes and check the running instances. In the screenshot in Figure 3, we have logged into both the nodes simultaneously, and the results can been seen in the figure.

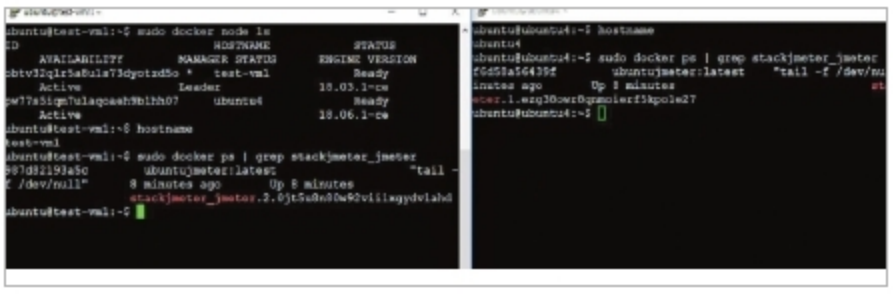

Figure 3: Checking how Docker instances are running

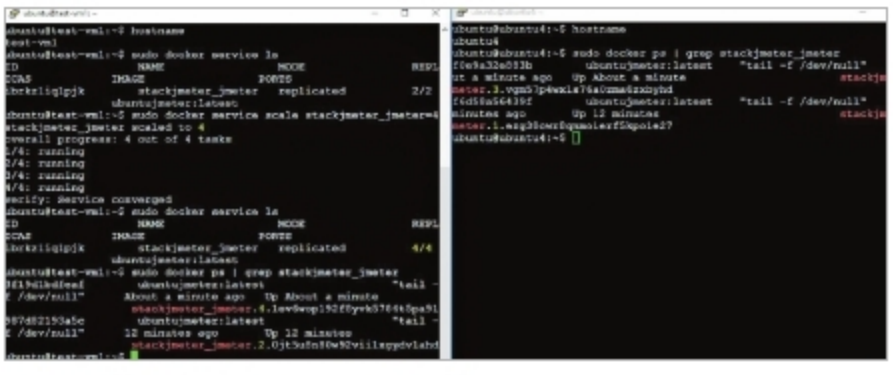

Figure 4: Scaling up Docker containers

Note: The one on the left hand side window is the master node running a Docker instance, and on the right hand side is the slave node running another Docker instance of the same image.